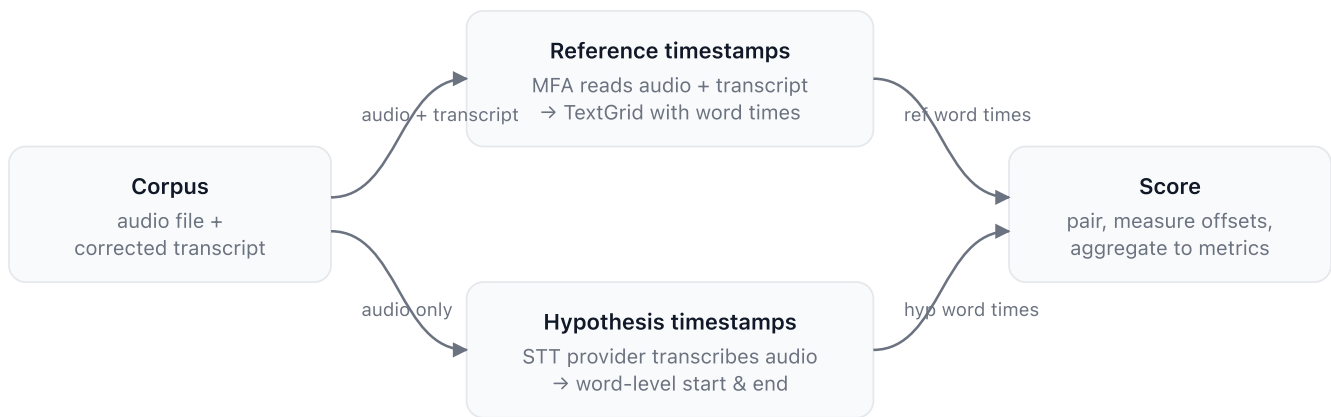


How this eval works

This page explains the eval methodology in plain language with concrete examples — what the harness does, what each metric means, what the results can and can't tell you, and where the inevitable noise comes from.

The pipeline at a glance



Two streams of word-level timestamps for the same audio — one from a forced aligner that "cheats" by being given the correct transcript, and one from the STT provider doing recognition from scratch. We line up the words and measure how far apart the times are.

A worked example, end to end

Let's walk through one real LibriSpeech utterance through every stage. The audio is 1.2 seconds long; the spoken transcript is:

"he doesn't work at all"

1. Reference from MFA

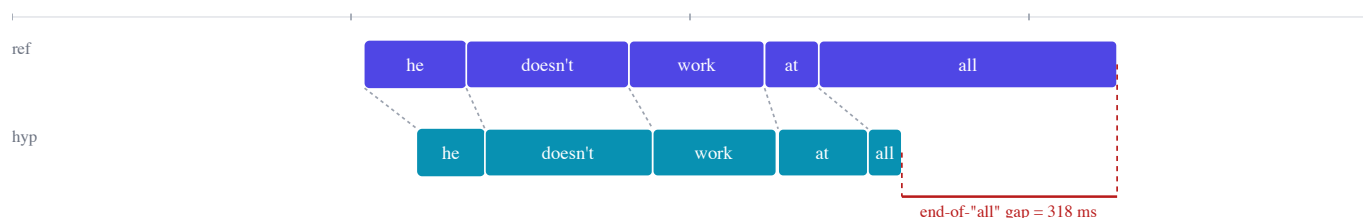
MFA takes the audio plus the correct transcript and aligns the two: it figures out exactly when each word's phonemes occur in the audio. The output is a TextGrid with one timestamped box per word. Drawn on a time axis:



■ MFA reference (this is the "ground truth")

2. Hypothesis from the STT provider

The same audio goes to the STT provider (here, AssemblyAI universal-3-pro). The provider produces its own word-level timestamps without seeing the ground-truth transcript. Drawn on the same time axis:



■ MFA reference ■ AAI hypothesis ■ notable boundary gap

Most word boundaries line up tightly. The exception is the end of "all" — MFA includes the trailing breath/silence in the word; the provider cut the word off earlier. **This isn't necessarily a model error** — it's a real disagreement about where the word ends. That ambiguity is intrinsic to word boundaries in continuous speech.

3. Aligning the two word sequences

Both sides have 5 words and they're in the same order, so pairing is straightforward: he ↔ he , doesn't ↔ doesn't , and so on. When the model mis-recognises words or the word counts differ, we use **minimum edit distance** alignment (the same algorithm WER uses) to find the best pairing. Substitutions still produce paired words — we score the timestamps regardless of whether the recognised text was right.

4. Measuring the offsets

Word	Ref start	Hyp start	Δ start	Ref end	Hyp end	Δ end
he	520	597	+77 ms	670	697	+27 ms
doesn't	670	697	+27 ms	910	946	+36 ms
work	910	946	+36 ms	1110	1129	+19 ms
at	1110	1129	+19 ms	1190	1262	+72 ms
all	1190	1262	+72 ms	1630	1312	-318 ms

Δ is the absolute offset: how far off the provider's word boundary is from MFA's. Sign just tells us "early vs. late." For metrics we take the absolute value.

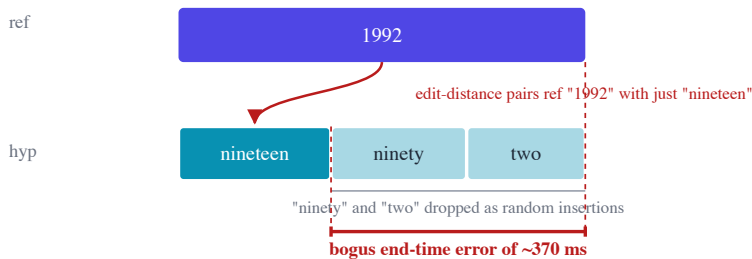
5. What this utterance contributes to the corpus metrics

This one utterance's local metrics: start MAE = $(77+27+36+19+72)/5 = 46$ ms · end MAE = $(27+36+19+72+318)/5 = 94$ ms. Notice how a *single* bad boundary ("all" end) doubles the end-MAE — this is exactly why we also report **median** (would have been 36 ms — much tighter) and **p90** (catches the outlier). The corpus-level metric is computed by pooling all word-pair offsets across every utterance, then computing MAE/median/p90/buckets once on the pooled set.

The multi-word substitution merge, visualized

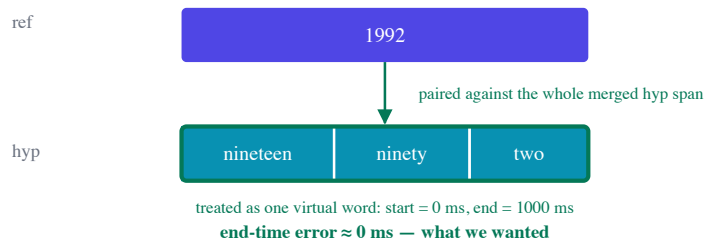
One word in the reference can map to multiple words in the hypothesis (or vice versa). The classic case is **inverse text normalization**: the reference transcript says "1992" but the provider, with `format_text` on, transcribed it as "nineteen ninety two". Without handling this, the harness would compare the timestamp of "1992" to the timestamp of just "nineteen", treating the other two words as random insertions — which produces *fake* timestamp errors.

Without merging (wrong)



Naïve edit-distance alignment pairs "1992" with just the first hypothesis word ("nineteen"), and treats "ninety" and "two" as random insertions that get dropped. The harness then compares the end-time of "1992" to the end-time of "nineteen", and concludes the model got the boundary 370 ms wrong — but the model didn't get anything wrong; it just spelled the number out.

With merging (right)



Whenever the alignment step finds an N-to-M substitution span, the harness merges the larger side into a single virtual word whose start is the first sub-word's start and whose end is the last sub-word's end. Now the comparison measures the full span the model emitted for that reference word, which is what we wanted in the first place.

What each metric actually means

MAE (mean absolute error)

What: the average distance — in milliseconds — between where the provider says a word starts/ends and where MFA says it does.

Why it can mislead: a single very-bad outlier pulls the average up a lot. The worked example above had four good word ends and one bad one, and MAE jumped from "tight" to "94 ms" because of that single outlier.

Median

What: the typical case — half the words are tighter than this, half are looser.

Why it matters: median is robust to outliers. If MAE is much bigger than median, the model has a few bad cases dragging the average up. That's a different problem than "the model is consistently a little off," and it calls for different fixes.

p90 (90th percentile)

What: the worst case, almost. 90% of words land tighter than this; 10% land looser.

Why it matters: if you're shipping a feature that depends on timestamps — subtitles, end-of-turn detection, voice-over editing — the worst 10% of words is what your users will notice. p90 is the realistic upper bound on "how bad will this feel."

% within X ms (tolerance buckets)

What: the fraction of words where the boundary lands inside a given tolerance. We report 25, 50, 100, and 200 ms.

Why it matters: these map directly to product decisions. Around 50 ms is roughly the limit of perceptual relevance for subtitle timing. 100 ms is a comfortable margin for response-trigger features. 200 ms is generous — anything outside that is "looks broken." Pick the bucket that matches what you're building.

Reading the tolerance buckets carefully

The buckets are **cumulative, not exclusive**. "% within 200 ms" means "the fraction of word boundaries that landed *anywhere from 0 to 200 ms* off the reference," not "the fraction in the 100–200 ms range specifically." A boundary that's only 30 ms off counts in the ≤ 50 , ≤ 100 , and ≤ 200 buckets simultaneously.

Concrete example using the LibriSpeech start-time numbers:

Bucket	AAI U3 Pro	Whisper-1	What it actually means
≤ 25 ms	18.9 %	18.5 %	fraction of starts within 25 ms of the reference
≤ 50 ms	43.2 %	41.0 %	within 50 ms (includes the ≤ 25 ms boundaries too)
≤ 100 ms	79.2 %	68.3 %	within 100 ms (includes both buckets above)
≤ 200 ms	99.3 %	83.9 %	within 200 ms (includes all of the above)

So "AAI wins at 200 ms" doesn't mean AAI is more often *in* the 100–200 ms range. It means AAI has fewer boundaries that miss by *more than* 200 ms — fewer catastrophic outliers. AAI's 99.3 % within 200 ms says only **0.7 %** of its word starts miss by more than 200 ms. Whisper's 83.9 % means **16.1 %** miss by more than 200 ms — about 22× as many catastrophically-bad starts. That's where Whisper's long p90 tail (310 ms) shows up in the bucket view.

Reading multiple buckets together tells the story

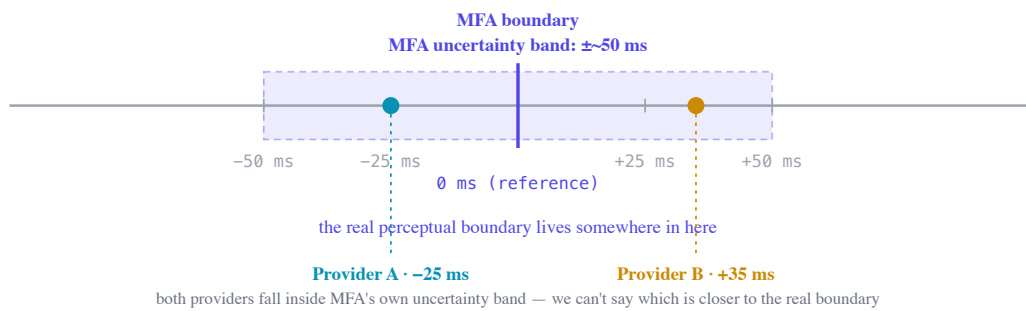
- **Wins at every bucket** → consistently better across the whole distribution.
- **Wins at tight (≤ 25 , ≤ 50) but loses at loose (≤ 200)** → very precise when it nails the timestamp, but has a long tail of catastrophic misses. (This is Whisper's pattern on peoples_speech — best ≤ 25 ms percentage, worst ≤ 200 ms.)
- **Wins at loose (≤ 200) but loses at tight (≤ 25)** → rarely catastrophic but rarely very precise either. Consistent, never great, never terrible.

If you actually want "% in the 100–200 ms range"

Subtract: `within_200` – `within_100` . For AAI on LibriSpeech, that's $99.3 - 79.2 = \mathbf{20.1\%}$ of boundaries fell strictly between 100 and 200 ms off. For Whisper, it's $83.9 - 68.3 = \mathbf{15.6\%}$. Useful if you care about a specific tolerance band rather than "anything tighter than X."

How small a difference is real?

MFA itself has alignment error. On clean read speech that's typically ~30–50 ms per boundary; on noisier or spontaneous speech, more. That's a **floor** on what these absolute numbers can resolve. Schematically:



The MFA boundary is the only point we actually know. The "real" perceptual boundary is somewhere in the dashed band, but we don't know exactly where. If two providers both land inside this band — even if they disagree with each other by 60 ms — we cannot conclude one is more accurate. The signal is below the noise floor of the reference. To resolve sub-30 ms differences you'd need a hand-annotated gold reference (or several MFA runs averaged with different acoustic models). **Rule of thumb for this corpus: treat MAE differences under ~30 ms as "tied."**

Why MFA, and what to use instead

MFA is the default reference aligner in this harness because three things line up: it's **open source and reproducible** (anyone can clone the harness and get the same TextGrids), it has the **highest published word-boundary accuracy** on clean read speech among the widely-used forced aligners, and it's **independent of the STT providers** we're scoring. That third point is the load-bearing one — if we used any neural-CTC-based aligner derived from the same architectures that the STT models use, the eval would systematically flatter providers whose acoustic models look like the reference aligner's.

On the published benchmark that matters here, Rousso et al. ([Interspeech 2024](#)) measured word-boundary accuracy across MFA, WhisperX, and MMS on TIMIT and Buckeye. MFA wins at every tolerance threshold; at ±10 ms it's at 41.6% vs WhisperX 22.4% and MMS 18.6%. That doesn't mean MFA is best in every regime — it means MFA is the right anchor for a defensible reference on clean-ish speech.

There are real reasons to pick something else though. The short version:

Aligner	Acoustic basis	When you'd pick it over MFA
MFA	Kaldi GMM-HMM (MFCC + LDA +	Default — highest published accuracy on clean speech in a supported language. Needs a transcript and a pronunciation dictionary.

Aligner	Acoustic basis	When you'd pick it over MFA
	SAT)	Pretrained models for ~30+ languages.
WhisperX	Whisper ASR + wav2vec2 CTC alignment	You <i>don't</i> have a reference transcript — WhisperX transcribes <i>and</i> aligns. Handles long-form natively (it VAD-chunks internally). Lower accuracy than MFA on word boundaries, so not ideal as a "gold" reference; useful as a quick check or for languages MFA doesn't support.
ctc-forced-aligner	Wav2Vec2 / HuBERT / MMS CTC	Your language isn't supported by MFA. MMS-based variant covers ~1130 languages. Handles long audio via 30-second sliding windows. Pip-installable, no conda dance.
NeMo Forced Aligner	NeMo CTC ASR (Conformer / Citrinet / QuartzNet)	You already run NeMo for ASR and want timestamps from the same acoustic model that produced your transcript. Heavy install (NeMo + PyTorch + CUDA). Officially tested only on English despite multilingual claims.
Aeneas	eSpeak TTS + MFCC DTW	You're sync'ing audiobook chapters or subtitle lines, not words. Trivially language-agnostic but word-level accuracy is unreliable.

If you swap MFA out, the harness still works — the eval driver parses MFA-style TextGrids from `--mfa-output`, so any aligner that emits Praat TextGrids (or that you convert to TextGrids) drops in. The methodology numbers will shift to be "relative to whatever-aligner-you-use." Be explicit about that change in any external write-up.

One nuance worth knowing about CTC-based aligners (WhisperX, ctc-forced-aligner, NFA): they exhibit "**peakiness**", where the CTC model puts almost all its probability on a single frame per token. Word *start* times are usually fine; word *end* times come out systematically less reliable than MFA's because the model marks the token at onset and emits blanks through the rest. There's active research on fixing this (e.g. [Huang et al., 2024 on label-prior CTC alignment](#)) but it's a real artifact you should know about if you're comparing the same audio across aligner families.

Long-form audio and why utterances are short

The harness expects your corpus to be pre-segmented into short utterances — seconds, not minutes. This isn't an arbitrary choice. Several things go wrong if you feed forced alignment a single 30-minute audio file:

1. **MFA's Viterbi beam search compounds error over long sequences.** Under the hood MFA runs Kaldi's `gmm-align-compiled` with a pruning beam (default 10, retry 40). On a 30-minute utterance there are ~180,000 frames. Small per-frame likelihood-ranking errors accumulate, the best-path likelihood drops below the worst-kept-hypothesis threshold, and Kaldi gives up with "beam too narrow." You can widen the beam (`--beam 400 --retry_beam`

1000), but the alignment quality drops significantly — you've just bought yourself a result, not a good one. MFA's docs explicitly recommend utterances under 30 seconds.

2. **Speaker adaptation needs short reliable utterances.** MFA's SAT/fMLLR pass estimates a per-speaker linear transform on the features. It assumes a stable speaker and microphone within each utterance. A long single utterance with mixed speakers or recording-condition changes blows that assumption up.
3. **The drift problem isn't MFA-specific.** Neural CTC aligners (WhisperX, ctc-forced-aligner, NFA) appear to "handle long-form natively," but only because they chunk internally — WhisperX VAD-segments to ~30 s, ctc-forced-aligner uses a 30 s sliding window with 2 s overlap, NFA streams CTC log-probs. The underlying drift in long-sequence alignment is a known issue across all aligner families; Rouso et al. (2024) recommend "recursive forced alignment" with multi-pass boundary refinement and observed drift on Buckeye utterances up to ~9 minutes long across every system they tested.

The standard long-form pipeline is straightforward:

1. Run a **VAD** ([silero-vad](#) or [pyannote/segmentation-3.0](#)) to find utterance boundaries — speech runs separated by silence.
2. Cut the audio at those VAD boundaries into utterances of roughly 5–30 seconds each.
3. Cut the transcript at the matching positions. This is the hard part — it usually requires either a rough timestamp from the VAD pass paired with sentence boundaries from the transcript, or a one-pass first alignment to bootstrap utterance-level cuts.
4. Run forced alignment per utterance independently.
5. Re-stitch the resulting TextGrids by adding each utterance's start offset back to the word times.

MFA ships a `mfa segment` command that wraps SpeechBrain VAD for step 1. WhisperX bakes the whole VAD-then-align flow into its tool. If you're starting from long meeting recordings (earnings calls, podcasts, depositions) and need timestamp evals on them, plan for the segmentation step — it's the longest pole in the corpus-prep work, and getting it right matters more for the numbers than the choice of aligner.

Target utterance length: ~5–30 seconds is the sweet spot for every aligner discussed above. Shorter than ~3 seconds loses context for HMM models; longer than ~30 seconds is where CTC peakiness and HMM beam pruning both start hurting.

What this eval can and can't tell you

Can tell you:

- Which provider's word boundaries match MFA's most consistently on this corpus.

- How the distribution of timestamp errors looks (tight vs. long-tailed).
- Whether the provider's timestamps are good enough for a downstream feature with a known tolerance budget.
- Relative performance across providers, fairly — because all providers are compared against the same reference.

Can't tell you:

- Absolute "ground-truth" accuracy in ms — you're bounded below by MFA's own error.
- How a provider will do on speech that's wildly different from this corpus (noisy, accented, conversational, non-English) — you'd run it on those datasets specifically.
- Whether the model recognised the right words. This is a timestamp eval, not a WER eval. The match-rate column tells you roughly what fraction of reference words were aligned, but for word-recognition quality you'd use a WER harness.

Glossary

Forced alignment

Given audio and a known transcript, computing exactly when each word/phoneme occurs in the audio. "Cheats" in the sense that it has the right transcript to start with — that's why MFA's output is a stronger reference than a recogniser's output.

MFA (Montreal Forced Aligner)

The open-source, widely-used forced aligner this harness uses for reference timestamps. [docs](#)

TextGrid

The file format MFA outputs — one file per audio clip, listing each word with its start and end times.

Word-level timestamps

For each word the STT model emitted, the start and end time in the audio. Some providers produce them natively from the recognition model; others run a separate alignment pass on the recognised text. Quality varies; that's what we're measuring.

Inverse text normalization (ITN)

The post-processing step that turns spoken numbers / dates / addresses into their written form:

"twenty twenty six" → "2026", "five dollars" → "\$5". It's what makes transcripts look professional and also what creates N-to-M alignment mismatches in this eval.

Edit distance / minimum-edit-distance alignment

The same algorithm WER uses: find the cheapest sequence of inserts, deletes, and substitutions that turns one word list into the other. The resulting alignment tells us which hypothesis word corresponds to which reference word.

Substitution merging

This harness's fix for N-to-M alignment chunks. When a reference word maps to several hypothesis words (or vice versa), merge the longer side into one virtual word spanning from the first sub-word's start to the last sub-word's end, then compare timestamps as a 1:1 pair.

MAE / median / p90

Three different ways to summarise a distribution of errors: the average, the typical, and the worst-realistic. We report all three because they tell different stories.

Tolerance bucket

"% of word boundaries that landed within X milliseconds of the reference." More directly useful for product decisions than averages.

Pooled aggregation

Computing the corpus-level metric by pooling every word-pair offset across every utterance into one big set, then computing once. This is the correct way to compute corpus-level percentiles — a weighted mean of per-file medians is not the same as the median of the pool.

p90 tail

The shape of the distribution past the 90th percentile. A long p90 tail means "the bad cases are very bad," which is often more annoying to users than "everything is consistently OK."

VAD (voice activity detection)

A model that finds the time spans where speech is happening in an audio file. Used to cut long recordings into utterance-sized chunks before forced alignment. [silero-vad](#) and [pyannote/segmentation-3.0](#) are the standard open-source choices.

CTC (connectionist temporal classification)

The output layer used by most modern non-Whisper neural ASR / aligner models (wav2vec2, HuBERT, MMS). It emits one prediction per audio frame plus a "blank" symbol, and the model learns to spike on a single frame per output token. Word boundaries derived from CTC have a known "peakiness" artifact — onsets are sharp, ends are unreliable.

HMM (hidden Markov model)

The classical acoustic-modelling framework MFA uses. Audio frames are emitted by a sequence of hidden phoneme states; the alignment is the most-probable state sequence (Viterbi). Slower to train, more robust to long sequences than CTC if the audio is well-segmented.

Beam search / beam pruning

The decoding algorithm that finds the best alignment path. At each frame, only the top-N most-likely partial paths are kept (the "beam"); the rest are pruned. MFA's default beam is 10 — small enough that long single utterances can lose the correct path off the bottom of the beam, which is why pre-segmentation matters.

fMLLR / speaker adaptation

A per-speaker linear feature transform that MFA estimates to compensate for differences in voices and microphones. Assumes a single stable speaker per utterance — another reason long mixed-speaker recordings should be cut before alignment.

Recursive forced alignment

A long-form alignment strategy where a first pass gives coarse word boundaries, then those boundaries are used to cut the audio for a refined second pass. Improves drift on long utterances at the cost of compute. ([Rousso et al., Interspeech 2024](#))